KOREATECH

주차별 총정리-1

# SQL 기초 총정리

## 1~6주차 SQL × 경영데이터의 이해 1~3장

한국기술교육대학교 경영학부

경영데이터베이스및실습 (CCT770) · 2026학년도 2학기

학습목표

# 총정리-1을 마치면

| | |
|---|---|
| 목표 ① | DBMS·키·관계로 다온유통 5개 표의 짜임을 설명한다 |
| 목표 ② | 업무 규칙을 제약조건(PK·FK·CHECK·DEFAULT)으로 옮긴다 |
| 목표 ③ | SELECT 실행 순서로 WHERE·GROUP BY·HAVING 자리를 고른다 |
| 목표 ④ | 경영 질문을 대상·지표·기간·비교 기준으로 쪼갠다 |
| 목표 ⑤ | 매출·객단가·취소율·재구매 같은 KPI를 SQL로 계산한다 |

**이론(1~2주) → 문법(3~6주) → 경영 질문(경영데이터 1~3장) → 실습 순서로 이어 갑니다**

지나온 길

# 1~6주차에 배운 것

| 1주차 | 2주차 | 3주차 | 4주차 | 5주차 | 6주차 |
|---|---|---|---|---|---|
| 데이터베이스 개요 · DBMS | 테이블 · 키 · 관계 · ERD | SQL 분류 · 데이터 타입 · DDL | INSERT · UPDATE · DELETE · 트랜잭션 | SELECT · WHERE · ORDER BY · 함수 | 집계 함수 · GROUP BY · HAVING |
| 남은 결과물 | 남은 결과물 | 남은 결과물 | 남은 결과물 | 남은 결과물 | 남은 결과물 |
| 실습 환경 · SQL 첫걸음 | 다온유통 ERD 읽기 | 5개 표 스키마 | 시드 데이터 적재 | 조건 검색 | 경영 통계 질의 |

경영데이터의 이해 1~3장(리터러시 · 기능별 데이터 · KPI)과 짝을 지어 다시 봅니다

목차

# 오늘의 흐름


1 이론 다시 보기 — DBMS · 키 · 관계

2 표 만들기와 넣기 — DDL · DML · 트랜잭션

3 조회의 문법 — SELECT 실행 순서 · 집계

4 경영데이터와 잇기 — 질문 · 데이터 발자국 · KPI

5 실습 — 질문에서 SQL로 (6문제)

6 자주 틀리는 지점 · 확인 퀴즈 · 핵심 요약

PART 1

# 이론 다시 보기

왜 데이터베이스인가, 표는 어떻게 이어지는가

# 파일로 관리하면 생기는 일

01

## 중복과 불일치

같은 고객 정보가 부서 파일마다
따로 있어 고치면 어긋난다

02

## 동시 접근

여러 사람이 같은 파일을
동시에 고치면 내용이 덮인다

03

## DBMS가 맡는 일

한곳에 모아 무결성 · 동시성 ·
보안 · 회복을 대신 지킨다

**경영데이터 1장: 데이터는 특정 부서의 기술이 아니라 모든 직무가 쓰는 회사의 공용어 — 그래서 한곳(DB)에 모은다**

2. 키의 종류

# 다온유통 표에서 찾은 키

| 키 | 뜻 | 다온유통 예 |
| --- | --- | --- |
| **기본키 (PK)** | 행을 하나로 식별 · NULL 불가 | customers.cust_id |
| **후보키** | 기본키가 될 수 있는 유일한 열 | customers.email — 단, NULL 허용이라 PK 불가 |
| **외래키 (FK)** | 다른 표의 기본키를 참조 | orders.cust_id → customers |
| **복합키** | 두 열을 묶어 하나의 기본키 | order_items (order_id, prod_id) |
| **자기참조 FK** | 같은 표의 기본키를 참조 | employees.manager_id → emp_id |

**기본키는 행을 하나로 짚고, 외래키는 다른 표의 기본키를 가리켜 표와 표를 잇는다**

2. 관계 — 1:N 과 M:N

# 다섯 개의 표는 이렇게 이어진다

| 관계 | 종류 | 잇는 키 |
|---|---|---|
| **고객 — 주문** | 1 : N | cust_id (고객 한 명이 여러 번 주문) |
| **직원 — 주문** | 1 : N | emp_id (NULL = 담당 없는 온라인 주문) |
| **주문 — 주문 상세** | 1 : N | order_id |
| **상품 — 주문 상세** | 1 : N | prod_id |
| **주문 — 상품** | M : N | order_items 가 둘을 연결 |
| **직원 — 직원** | 1 : N | manager_id (팀장 — 팀원) |

**주문(M) : 상품(N) 관계는 연결 표 order_items 로 풀어 두 개의 1:N 으로 만든다**

3. SQL 의 분류

# 명령은 네 갈래로 나뉜다

| 갈래 | 명령 | 하는 일 |
| --- | --- | --- |
| DDL 정의 | CREATE · ALTER · DROP · TRUNCATE | 표의 구조를 만든다 |
| DML 조작 | SELECT · INSERT · UPDATE · DELETE | 행을 읽고 바꾼다 |
| DCL 제어 | GRANT · REVOKE | 권한을 준다 · 거둔다 |
| TCL 트랜잭션 | START TRANSACTION · COMMIT · ROLLBACK | 묶어서 확정 · 취소 |

> 3~4주차가 DDL · DML · TCL, 10주차가 DCL 이다

# CREATE TABLE products

```
1  CREATE TABLE products (
2    prod_id   CHAR(4)      PRIMARY KEY,
3    name      VARCHAR(50)  NOT NULL,
4    category  VARCHAR(20)  NOT NULL,
5    price     INT          NOT NULL CHECK (price >= 0),
6    stock     INT          NOT NULL DEFAULT 0
7  );
```

**2행**

**1 PRIMARY KEY**

'P001' 형식, 중복 · NULL 금지

**3행**

**2 NOT NULL**

상품명은 반드시 있어야 한다

**5행**

**3 CHECK**

음수 가격은 DB 가 거부한다

**6행**

**4 DEFAULT 0**

재고를 안 넣으면 0 (품절)

> 규칙을 화면이 아니라 스키마에 박아 두면 어떤 프로그램이 넣어도 같은 규칙이 지켜진다

4. DDL — 업무 규칙을 스키마에

# 업무 규칙 → 제약조건

| 업무 규칙 | 제약조건 | 어긋나면 |
|---|---|---|
| **신규 고객은 SILVER 등급** | grade DEFAULT 'SILVER' | 값을 안 넣어도 SILVER |
| **이메일은 중복 금지** | email UNIQUE | ERROR 1062 중복 |
| **주문 수량은 1개 이상** | qty CHECK (qty > 0) | ERROR 3819 CHECK 위반 |
| **없는 고객의 주문 금지** | FOREIGN KEY (cust_id) | ERROR 1452 참조 위반 |
| **한 주문에 같은 상품 두 줄 금지** | PRIMARY KEY (order_id, prod_id) | ERROR 1062 중복 |

**경영데이터 2장: 데이터 품질은 발생 · 기록 단계에서 결정된다 — 제약조건이 그 입구를 지킨다**

# 전부 성공 아니면 전부 취소

```
1  START TRANSACTION;
2  UPDATE products SET stock = stock - 2
3  WHERE  prod_id = 'P001';
4  ROLLBACK;
5
6  SELECT prod_id, name, stock
7  FROM   products WHERE prod_id = 'P001';
```

**1** 1행
**묶음 시작**
이후 변경은 아직 확정 전

**2** 2–3행
**WHERE 를 꼭**
빼면 모든 상품의 재고가 준다

**3** 4행
**ROLLBACK**
묶음 안의 변경을 모두 되돌림

**4** 6–7행
**확인**
P001 재고 25 그대로

> **주문 접수 · 재고 차감 · 결제처럼 함께 성공해야 하는 일은 하나의 트랜잭션으로 묶는다 (확정은 COMMIT)**

PART 2

# 조회의 문법

SELECT 는 쓰는 순서와 실행 순서가 다르다

6. SELECT 실행 순서

# DB 가 실제로 읽는 순서

## 쓰는 순서는 SELECT 가 먼저지만, 실행은 FROM 부터

| 01 | 02 | 03 | 04 |
|---|---|---|---|
| FROM | WHERE | GROUP BY | HAVING |
| 어느 표에서 | 행을 거른다 | 묶는다 | 묶음을 거른다 |

| 05 | 06 | 07 |
|---|---|---|
| SELECT | ORDER BY | LIMIT |
| 열 · 계산 · 별칭 | 정렬 | 몇 행만 |

**SELECT 의 별칭은 WHERE 에서 쓸 수 없다(아직 만들어지기 전) — ORDER BY 에서는 쓸 수 있다**

6. 조건의 자리

# WHERE · HAVING · NULL

01

## WHERE

집계 전에 행을 거른다

WHERE status <> '취소'

02

## HAVING

집계 후에 그룹을 거른다

HAVING COUNT(*) >= 2

03

## NULL

= NULL 이 아니라 IS NULL

COUNT(email) 은 NULL 을 뺀다

**집계 함수(SUM · COUNT · AVG)가 들어간 조건은 HAVING, 행 하나로 판정할 수 있는 조건은 WHERE**

PART 3

# 경영데이터의 이해와 잇기

질문을 만드는 훈련(1~3장)과 답을 꺼내는 도구(SQL)

# 질문의 요소가 SQL 의 부품이 된다

| 질문의 요소 | SQL 의 자리 | 예 |
|---|---|---|
| **대상 — 누구 · 무엇** | FROM · WHERE | FROM orders WHERE status <> '취소' |
| **지표 — 무엇을 잴까** | SELECT 집계 함수 | SUM(qty * unit_price) AS 매출액 |
| **기간 — 언제** | WHERE 날짜 조건 | order_date BETWEEN '2026-01-01' AND '2026-06-30' |
| **비교 기준 — 무엇과** | GROUP BY · ORDER BY | GROUP BY category ORDER BY 매출액 DESC |

**예: 「2026년 상반기 카테고리별 매출(취소 제외)을 비교하면?」 — 대상 · 지표 · 기간 · 비교 기준이 모두 있다**

# 주문 한 건이 남기는 기록

## 한 번의 주문이 여러 부서의 표로 퍼지고, 키가 그 흐름을 잇는다

| 01 주문 접수 | 02 상품 내역 | 03 재고 차감 | 04 담당 실적 | 05 고객 등급 |
|---|---|---|---|---|
| orders 1행 (영업) | order_items N행 (영업 · 재무) | products.stock (물류) | employees (인사 · 영업) | customers (마케팅) |

**부서마다 보는 표는 달라도 뿌리는 같은 주문 — 그래서 order_id · cust_id · prod_id 로 이어 볼 수 있다**

# 「매출」은 정의부터 정한다

같은 15건의 주문인데 기준 시점에 따라 숫자가 달라진다

**1,031,600**

**취소 포함 주문액**

주문 15건 전부

**965,100**

**취소 제외 매출**

취소 2건(66,500원) 뺌

**807,400**

**배송완료 기준**

배송완료 9건만

다온유통 실습 DB · 2026년 1~6월 · 단위 원 — 부서 간 숫자 차이는 계산 오류가 아니라 정의 차이일 수 있다

# KPI 를 SQL 로 재기

| 지표 | 무엇을 보나 | SQL |
|---|---|---|
| **매출액** | 성장 | SUM(qty * unit_price) |
| **주문 건수** | 성장 | COUNT(DISTINCT order_id) |
| **객단가** | 성장 | 매출액 / 주문 건수 |
| **재구매 고객** | 고객 | GROUP BY cust_id HAVING COUNT(*) >= 2 |
| **취소율** | 운영 | SUM(CASE WHEN status = '취소' THEN 1 ELSE 0 END) / COUNT(*) |
| **품절 상품** | 운영 | WHERE stock = 0 |

**지표는 DB 에 기록된 것만 잴 수 있다 — 지표 설계와 데이터 설계는 한 몸이다**

PART 4

# 실습 — 질문에서 SQL로

웹 실습 화면 또는 내 MySQL(daon)에서 직접 실행합니다

실습 1 · 상품 구성

# 카테고리별 상품 수 · 평균가 · 재고

실습

```
1  SELECT category         AS 카테고리,
2         COUNT(*)         AS 상품수,
3         ROUND(AVG(price)) AS 평균가,
4         SUM(stock)       AS 총재고
5  FROM   products
6  GROUP  BY category
7  ORDER  BY 평균가 DESC;
```

2행
**1 COUNT(*)**
그룹 안의 행 수

3행
**2 ROUND(AVG)**
평균가를 정수로

6행
**3 GROUP BY**
카테고리마다 한 행

결과: 가전 2종 115,500원 · 식품 3종 19,433원 · 생활용품 3종 9,733원 · 문구 2종 3,850원 (재고는 문구 380개로 가장 많다)

실습 2 · 성장 KPI

# 주문 건수 · 주문액 · 객단가

실습

```sql
SELECT COUNT(DISTINCT order_id)          AS 주문건수,
       SUM(qty * unit_price)             AS 주문액,
       ROUND(SUM(qty * unit_price)
             / COUNT(DISTINCT order_id)) AS 객단가
FROM   order_items;
```

1행

**1 DISTINCT**

상세 25행 → 주문 15건

2행

**2 수량 × 주문 시점 단가**

products.price 가 아니다

3–4행

**3 객단가**

주문 1건당 평균 금액

결과: 15건 · 1,031,600원 · 객단가 68,773원 — 그런데 취소 2건이 섞여 있다. 거르려면 orders.status 가 필요 → 총정리-2(JOIN)

실습 3 · 운영 KPI

# 주문 상태별 건수와 취소율

실습

```sql
SELECT status AS 상태, COUNT(*) AS 건수
FROM   orders
GROUP  BY status
ORDER  BY 건수 DESC;

SELECT COUNT(*) AS 전체주문,
       SUM(CASE WHEN status = '취소' THEN 1 ELSE 0 END) AS 취소,
       ROUND(SUM(CASE WHEN status = '취소' THEN 1 ELSE 0 END)
             * 100.0 / COUNT(*), 1) AS 취소율
FROM   orders;
```

**3행**

**1 상태마다 한 행**

배송완료 9 · 배송중 3 · 취소 2 · 주문접수 1

**7행**

**2 조건부 개수**

취소면 1, 아니면 0 을 더한다

**8–9행**

**3 100.0 을 곱한다**

정수 나눗셈을 피해 % 로

결과: 15건 중 취소 2건 · 취소율 13.3% — 경영데이터 3장의 운영 지표(반품 · 취소율)

실습 2~3 · 잰 결과

# 지금까지 잰 다온유통 KPI

6주차 문법만으로도 회사의 상태를 숫자로 말할 수 있다

**15건**

**주문 건수**

취소 2건 포함

**68,773원**

**객단가**

취소 포함 기준

**13.3%**

**취소율**

15건 중 2건

> **경영데이터 3장: 모든 숫자에는 비교 기준을 붙인다 — 다음 달 · 같은 업종 · 목표와 견주어 읽는다**

실습 4 · 성장 추이

# 월별 주문 건수 (취소 제외)

실습

```
1  SELECT DATE_FORMAT(order_date, '%Y-%m') AS 월,
2         COUNT(*)                         AS 주문건수
3  FROM   orders
4  WHERE  status <> '취소'
5  GROUP  BY DATE_FORMAT(order_date, '%Y-%m')
6  ORDER  BY 월;
```

1행

**1 DATE_FORMAT**

날짜를 '2026-01' 같은 월로

4행

**2 WHERE 가 먼저**

취소를 빼고 묶는다

5행

**3 같은 식으로 묶기**

월마다 한 행

결과: 1월 2 · 2월 2 · 3월 3 · 4월 2 · 5월 2 · 6월 2 — 전월 대비 성장률은 (이번 달 − 지난달) ÷ 지난달

실습 5 · 고객 KPI

# 두 번 이상 산 고객 (재구매)

실습

```sql
SELECT cust_id  AS 고객번호,
       COUNT(*) AS 주문횟수
FROM   orders
WHERE  status <> '취소'
GROUP  BY cust_id
HAVING COUNT(*) >= 2
ORDER  BY 주문횟수 DESC;
```

1

2

4행

**1 WHERE**

행 조건 — 취소 주문 빼기

6행

**2 HAVING**

그룹 조건 — 2회 이상만

**결과: C001 3회 · C004 3회 · C002 2회 → 실제로 산 고객 8명 중 3명, 재구매율 37.5%**

실습 6 · 고객 등급

# 등급별 고객 수와 평균 포인트

실습

```
1  SELECT grade              AS 등급,
2         COUNT(*)           AS 고객수,
3         ROUND(AVG(points)) AS 평균포인트
4  FROM   customers
5  GROUP  BY grade
6  ORDER  BY 평균포인트 DESC;
```

1
2

2행
**1 등급마다 몇 명**
VIP 2 · GOLD 3 · SILVER 5

3행
**2 평균 포인트**
4,850 · 2,100 · 340

다음 질문: 「VIP 2명이 매출의 얼마를 차지하나?」 — 고객 표와 주문 표를 이어야 답할 수 있다 → 총정리-2

정리

# 자주 틀리는 지점

| 증상 | 원인 | 해결 |
| --- | --- | --- |
| ERROR 1111: Invalid use of group function | WHERE 에 SUM · COUNT 를 씀 | 그룹 조건은 HAVING 으로 |
| WHERE email = NULL 이 0행 | NULL 은 = 로 비교되지 않음 | IS NULL / IS NOT NULL |
| ERROR 1054: Unknown column '할인가' | WHERE 에서 SELECT 별칭을 씀 | 식을 그대로 쓰거나 ORDER BY 에서만 |
| UPDATE 뒤 모든 행이 바뀜 | WHERE 를 빠뜨림 | 같은 WHERE 로 먼저 SELECT 해 보고, 트랜잭션 안에서 |
| ERROR 1452: foreign key constraint fails | 부모 표에 없는 값을 넣음 | 부모(고객 · 상품)를 먼저 INSERT |

# 확인 퀴즈

**카테고리별 평균 판매가를 구한 뒤, 평균가가 10,000원 이상인 카테고리만 남기려 합니다. 조건을 쓸 곳은?**

① WHERE AVG(price) >= 10000

② HAVING AVG(price) >= 10000

③ ORDER BY AVG(price) >= 10000

④ LIMIT 10000

확인 퀴즈

# 정답 및 해설

정답

## ② HAVING AVG(price) >= 10000

해설

평균가는 GROUP BY 로 묶은 뒤에야 계산되므로, 집계 결과에 거는 조건은 HAVING 에 씁니다. WHERE 는 묶기 전 행 하나하나를 거르는 자리라 집계 함수를 쓰면 ERROR 1111 이 납니다. 결과는 가전(115,500원) · 식품(19,433원) 두 카테고리입니다.

정리

# 핵심 요약

| | |
|---|---|
| 핵심 ① | DBMS 는 흩어진 데이터를 모아 무결성 · 동시성 · 보안을 지킨다 |
| 핵심 ② | 기본키로 행을 짚고, 외래키로 표를 잇고, M:N 은 연결 표로 푼다 |
| 핵심 ③ | 업무 규칙은 제약조건으로 스키마에, 함께 할 변경은 트랜잭션으로 |
| 핵심 ④ | 실행 순서 FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY |
| 핵심 ⑤ | 경영 질문 = 대상 · 지표 · 기간 · 비교 기준, 「매출」은 정의부터 |

**같은 데이터라도 정의(취소 포함 · 제외 · 배송완료)를 먼저 정해야 숫자가 하나로 모인다**

다음 총정리 예고

# 총정리-2 · 연결 · 비교 · 운영

01

## JOIN

표를 이어

취소를 거른 매출

02

## 서브쿼리

평균 · 전체와

비교하는 질문

03

## 뷰 · 인덱스

대시보드를

이름 하나로

04

## 권한

고객 정보는

필요한 만큼만

KOREATECH

주차별 총정리-1 마무리

# 학습 마무리

오늘 다시 본 문법을 경영 질문으로 바꿔 직접 실행해 봅니다

- ✓ 키 · 관계 · 제약조건
- ✓ SELECT 실행 순서와 WHERE · HAVING
- ✓ 좋은 데이터 질문과 매출의 정의
- ✓ KPI 를 SQL 로 재기

이어지는 활동

**총정리-1 페이지 실습 6문제**